

general

Initial prompt

I want to build an AI-capable bot for Discord and for the moment I need to understand how to think about this.

I want to use a local computer running a local open source AI model, and running Discord at the same time, connected to a Discord server, to provide AI assistance to members of this Discord server and making some inferences about the activity on the server and providing general overview and support.

I want to design the bot in a way that I can swap AI models for testing and choosing the best ones for the case. I plan to start experimenting with Qwen2.5, Gemma, Mistral 7B, Phi-4 small, and DeepSeek distilled.

I also want to know more about AI infrastructure, because in order to make the bot more responsive, I want the bot to perform scheduled work, such as semantic indexing or RAG, building various databases for specific uses, scanning external resources, perhaps using MCP servers, and harness to constraint the behaviour of the model. You can suggest other infrastructure components.

I also want to understand how to install the AI model and everything around it (the infrastructure), how to install the Discord bot and to connect it to the local AI infrastructure and how to connect the bot to Discord.

Please give me a plan with ample information about every element for me to understand the what, why and how.

AI infrastructure and implementation brainstorm - [link to Grok discussion](#)

High-level architecture plan: Run everything locally on your computer for privacy, zero API costs, and full control. Use **Ollama** as the core for running and swapping LLMs easily. Pair it with **discord.py** for the bot. Add **LangChain** or **LlamaIndex** + a vector DB (e.g., **Chroma**) for RAG and semantic capabilities. Use Python scheduling (APScheduler) or cron for background tasks.

Core Components & Why They Matter

- **Ollama (LLM Runner):** Simplest way to run open-source models locally. It provides a local HTTP server (OpenAI-compatible API at `http://localhost:11434`), easy model

management (ollama pull/run), and Modelfiles for customization (system prompts, behavior constraints). Why? Model swapping is trivial—just change the name in code/config. Supports your list: Qwen2.5, Gemma (e.g., gemma2/gemma3), Mistral, Phi (phi3/phi4), DeepSeek variants.

- **Discord Bot (discord.py)**: Python library for Discord API interaction. Handles events (messages, mentions), commands, and server activity listening. Why? Mature, async-friendly, and easy to integrate with local LLM calls.
- **RAG & Vector Store (LlamaIndex/LangChain + Chroma)**: For semantic search over server history, docs, or external resources. Embed documents/chats into vectors, retrieve relevant context before LLM generation. Why? Prevents hallucination; enables server-specific knowledge (e.g., "summarize recent activity in #general").
- **Scheduling & Background Tasks**: APScheduler (Python-native) or system cron. Why? Periodic indexing, summaries, scans without blocking the bot.
- **Databases**: SQLite/Postgres for chat history/metadata; Chroma (or pgvector) for vectors. Optional: Redis for caching/rate limiting.
- **MCP (Model Context Protocol)**: Emerging standard for connecting LLMs to tools/data sources securely (files, DBs, GitHub, etc.). Why? Standardized way to give your bot "tools" beyond basic RAG—e.g., query external resources or constrain actions.
- **Other suggestions**:
 - **Open WebUI** or similar for a web dashboard to test models/chat.
 - **LM Studio** as Ollama alternative if you prefer GUI.
 - **Guardrails** or simple prompt engineering + Modelfiles for behavior constraints.
 - Monitoring: Prometheus/Grafana or simple logging for resource usage (important on local hardware).

Hardware considerations: Smaller quantized models (e.g., 7B Q4/Q5) run well on CPU + 16+ GB RAM. GPU (NVIDIA with CUDA, or AMD) accelerates significantly. Start with smaller variants for testing.

Installation & Setup

Ollama + Models

- ☒ ~~Install Ollama (cross-platform):~~
 - ☐ macOS/Linux: `curl -fsSL https://ollama.com/install.sh | sh`
 - ☐ Start service: `ollama serve` (runs in background).
- ☒ ~~Pull models (run in terminal)~~

```
ollama pull qwen2.5:7b-instruct
ollama pull gemma2:9b
ollama pull mistral
ollama pull phi4 # or phi3.5-mini etc.
```

```
ollama pull deepseek-r1 # or relevant variant
```

Test: ollama run qwen2.5:7b-instruct (interactive chat). Use tags like :q4_0 for quantization to save RAM.

- ☐ For custom behavior: Create a **Modelfile** (e.g., for constraints/personality):
text

```
FROM qwen2.5:7b
```

```
SYSTEM ""You are a helpful Discord assistant. ... [rules for inferences, safety]""
```

```
ollama create my-assistant -f Modelfile
```

Discord Bot

- ☒ ~~Go to Discord Developer Portal → New Application → Bot → Create Bot. Copy token. Enable "Message Content" Intent. Generate OAuth2 invite URL with bot + applications.commands-scopes, permissions (Send Messages, Read Message History, etc.). Invite to server.~~
- ☒ ~~Python environment (use virtualenv/conda):~~

```
pip install discord.py ollama python-dotenv langchain langchain-community langchain-ollama  
llama-index chromadb apscheduler
```

Basic Bot + LLM Integration Example

Create bot.py (adapt from tutorials):

Python

```
import discord  
import os  
from dotenv import load_dotenv  
import ollama # or requests to localhost:11434  
  
load_dotenv()  
TOKEN = os.getenv('DISCORD_TOKEN')  
MODEL = "qwen2.5:7b-instruct" # Easy swap  
  
intents = discord.Intents.default()  
intents.message_content = True  
client = discord.Client(intents=intents)  
  
@client.event  
async def on_ready():  
    print(f"Logged in as {client.user}")
```

```

@client.event
async def on_message(message):
    if message.author == client.user or not message.content.startswith('!ai '):
        return
    prompt = message.content[4:]
    response = ollama.chat(model=MODEL, messages=[{'role': 'user', 'content': prompt}])
    await message.channel.send(response['message']['content'])

client.run(TOKEN)

```

Run with `python bot.py`. Extend for mentions, threads, etc. Use ollama Python lib or HTTP for streaming.

Model swapping: Config file/env var for MODEL. Or dynamic via commands.

AI Infrastructure for Responsiveness & Advanced Features

- **RAG Pipeline** (using LlamaIndex + Chroma + Ollama):
 - Load documents/server history → chunk → embed (Ollama embedding model or nomic-embed) → store in Chroma.
 - Query: Retrieve top-k relevant → augment prompt → generate. Example setup: Many tutorials combine LlamaIndex with Ollama embeddings/LLM.

- **Scheduled Tasks** (e.g., index new messages, build summaries, scan resources): Use **APScheduler**:

Python

```

from apscheduler.schedulers.asyncio import AsyncIOScheduler
scheduler = AsyncIOScheduler()
scheduler.add_job(index_server_activity, 'cron', hour=2) # Daily at 2 AM

scheduler.start()

```

Tasks: Fetch recent messages (with rate-limit awareness), embed & upsert to vector DB, generate overviews.

- **Databases:**
 - Vector: Chroma (simple, local files) or pgvector (if using Postgres).
 - History: SQLite for persistence/conversations.
- **MCP & Tools:** Implement or use existing MCP servers for file access, DB queries, etc. This lets the LLM call tools in a standardized way. Great for "scanning external resources."
- **Constraints/Harness:**

- System prompts + JSON mode/output parsing.
 - LangChain agents with tool calling.
 - Moderation layer before/after generation.
 - Rate limiting in bot (Discord + your own) to avoid overload.
- **Server Activity Inferences:** Log messages (respect privacy), embed periodically, query for summaries/trends ("what's the vibe in the server?").

Full Development & Deployment Plan

1. **Prototype:** Basic bot + Ollama chat. Test model swapping.
2. **Add RAG:** Index server docs/channels. Test queries.
3. **Background Jobs:** Scheduler for indexing/summaries.
4. **Advanced:** MCP/tools, conversation memory (store history vectors), slash commands, rate limiting/queuing, web UI.
5. **Testing:** Different models on same hardware. Monitor RAM/GPU with nvidia-smi or similar.
6. **Productionizing (local):** Run as systemd service (Linux) or always-on script. Use Docker for Ollama + bot.
7. **Scaling/Iteration:** If hardware limits, quantize more or offload to another machine (still local network). Add logging/monitoring.

Security/Privacy: Everything local = great. Still, be careful with message logging (GDPR-like considerations). Use Discord intents minimally.

Resources to Start:

- Ollama docs + library.
- discord.py quickstart.
- LlamaIndex/Ollama RAG tutorials.
- GitHub repos for Ollama Discord bots (search "ollama discord bot").

This gives modularity, responsiveness via background work, and a solid foundation. Start small, iterate by swapping models and adding one feature at a time. If you hit hardware snags or need code for a specific part, provide more details!

python env

Python Environment (virtualenv or conda)

It creates an isolated "box" for your project's Python packages. This prevents conflicts with other Python projects on your computer and keeps everything clean and reproducible.

Using `venv` (built-in, recommended for most people)

1. Open a terminal / command prompt.
2. Navigate to the folder where you want to create your bot project:

```
bash
mkdir discord-ai-bot
cd discord-ai-bot
```

3. Create the virtual environment:

```
```bash
python -m venv venv
```
```

(This creates a folder called `venv` inside your project.)

4. **Activate** the environment:

```
```bash
source venv/bin/activate
```
```

You should see *venv* appear at the beginning of your terminal prompt.

5. Upgrade pip (good practice):

```
```bash
pip install --upgrade pip
```
```

2. Installing the Required Packages

After activating your environment, run this command:


```
```bash
pip install discord.py ollama python-dotenv langchain langchain-community
langchain-ollama llama-index chromadb apscheduler
```
```

What each package does (briefly):

- discord.py — The main library to connect to Discord
- ollama — Python client to talk to your local Ollama models
- python-dotenv — Loads your secret tokens from a .env file (safer than hardcoding)
- langchain, langchain-community, langchain-ollama — Framework for building agents, RAG, and connecting to Ollama
- llama-index — Alternative/high-level RAG tools
- chromadb — Local vector database for semantic search / RAG
- apscheduler — For running background scheduled tasks (indexing, summaries, etc.)

Extra useful packages you may want to add later:

```
```bash
pip install langgraph langgraph-checkpoint-sqlite fastmcp # For advanced agents + MCP
```
```

Full Step-by-Step Setup (Recommended Order)

- ☐ Install Ollama from <https://ollama.com> and run it.
- ☐ Create project folder → create and activate virtual environment (as shown above).
- ☐ Run the pip install command.
- ☐ Create a .env file in your project root:

```
```
DISCORD_TOKEN=your_discord_bot_token_here
```
```

5. Start writing code (e.g. bot.py).

To verify everything works:

```
```bash
python -c "import discord; import ollama; import langchain; print('All packages imported successfully!')"
```
```

Daily Workflow Reminder

- Every time you open a new terminal to work on the project, you must ****activate the environment**** first:- macOS/Linux:

```
source venv/bin/activate
```

- When done, you can deactivate with

```
deactivate
```

mcp server

Implementing MCP servers locally integrates perfectly with your local Discord AI bot setup. MCP (Model Context Protocol) is an open standard (introduced by Anthropic) that lets your LLM securely connect to tools, data sources, and resources (files, DBs, Git, etc.) via a standardized interface.

This is more powerful and structured than basic function calling or raw RAG for giving your bot "hands" (e.g., read server logs, query a knowledge base, fetch external data safely, or constrain actions).

Why MCP for Your Bot?

- Tool Use & Context: LLM can discover and call capabilities (resources for read-only data, tools for actions, prompts for templates) dynamically.
- Local-First: Run servers on the same machine as Ollama and the bot. No cloud dependency.
- Modularity: Swap or add servers (e.g., filesystem for docs, Discord-specific for activity, vector DB queries).
- Security: Scoped permissions, read-only options, and standardized auth.
- Complements Your Stack: Use with Ollama (via compatible clients) + LangChain/LlamaIndex for enhanced RAG/agent behavior.

Quick Start: Using Existing MCP Servers Locally

The official repo has ready-to-run servers:

<https://github.com/modelcontextprotocol/servers>.

Install Prerequisites: Python servers: Use uv (recommended) or pip.

```
pip install uv # or follow uv docs
```

Node.js for TS servers (optional).

Run Examples:

- Filesystem: `uvx mcp-server-filesystem` (or similar).
- Git: `uvx mcp-server-git`.
- Memory, Postgres, etc.

Many run via `npx` (Node) or Python entrypoints. Start one and note its transport (stdio, SSE, or HTTP).

Connect to Your LLM/Bot:

- Ollama itself may need a wrapper/client that supports MCP (or use LangChain agents with MCP tools).
- For testing: Use Claude Desktop (has built-in local MCP support) or build a simple Python client.
- In your Discord bot: Expose MCP servers locally; have the bot's LLM orchestration layer connect to them (e.g., via mcp SDK).

Building a Custom MCP Server (Python Recommended)

FastMCP is the go-to library for Python—simple decorators for tools/resources.

```
mkdir my-mcp-server && cd my-mcp-server
uv init # or python -m venv
uv add fastmcp # or pip install fastmcp
```

Basic Example (server.py)

```
from mcp.server.fastmcp import FastMCP
from typing import List, Dict
import datetime

mcp = FastMCP("DiscordHelper") # Name your server

# Example Tool: Analyze recent server activity (mock or integrate with Discord API)
@mcp.tool()
async def get_server_overview(channel: str = "general", hours: int = 24) -> str:
    """Get a summary of recent activity in a Discord channel."""
    # Here: Query your DB/vector store or Discord history
    return f"Overview for #{channel} (last {hours}h): High activity on topic X..."

# Example Resource: Read-only knowledge base
@mcp.resource()
def get_knowledge_base() -> Dict:
    """Static or dynamic knowledge for the server."""
    return {"rules": ["Be helpful", "Respect privacy"], "faq": [...]}

# Prompts (optional templates)
@mcp.prompt()
def discord_response_template(user_query: str, context: str) -> str:
    return f"User asked: {user_query}\nContext: {context}\nRespond helpfully..."
```

```
if __name__ == "__main__":  
    mcp.run(transport="stdio") # Or "sse" for HTTP
```

Run: `uv run server.py` (or `python server.py`).

Key Components

- **Tools** (@mcp.tool): Actions (e.g., query DB, scan files, generate report). Can be async.
- **Resources** (@mcp.resource): Read-only data (e.g., vector search results, docs).
- **Prompts**: Reusable templates.
- **Transport**: stdio (simple for local), SSE/HTTP for broader access.

For Discord Integration:

- Tool to fetch message history (respect rate limits).
- Resource exposing vector DB (Chroma) embeddings for RAG.
- Scheduled indexing could push data to an MCP resource.

Connecting to Your Discord Bot + Ollama

1. Run MCP server(s) alongside Ollama and bot (use screen, systemd, Docker Compose, or supervisor).
2. In bot code (using LangChain or custom):
 - Discover MCP servers (config with URLs or stdio).
 - Use MCP client SDK to let the LLM call tools/resources.
 - Example flow: User message → Bot retrieves context via MCP → Augment prompt to Ollama → Respond.

For full agentic behavior, integrate with LangChain's tool-calling and MCP adapters if available.

Advanced Local Setup & Best Practices

- **Docker Compose** for everything:

yaml

```
services:  
  ollama:  
    image: ollama/ollama  
  mcp-server:  
    build: ./my-mcp-server  
  discord-bot:
```

build: .

- **Discovery & Config:** Use a central config file listing MCP endpoints. Many clients auto-discover via stdio or a manifest.
- **Security:** Start with read-only tools. Use environment variables for secrets. Limit scopes.
- **Testing:** Use MCP Inspector (community tool) or simple clients. Test with your models (Qwen2.5 etc. via Ollama).
- **Scaling Background Work:** Your scheduler can update MCP resources (e.g., refresh knowledge base nightly).
- **Resources:**
 - Official servers: <https://github.com/modelcontextprotocol/servers>
 - Python template: Search for create-mcp-server or FastMCP examples.
 - Guides: modelcontextprotocol.io, GitHub repos for beginners.

Potential Challenges & Tips

- **LLM Compatibility:** Not all local models handle tool calling as well as larger ones. Test with Qwen2.5 or Mistral; fallback to prompt engineering.
- **Performance:** Keep servers lightweight. Run on same machine for low latency.
- **Debugging:** Logs are key. Start with simple tools.
- **Extensions:** Build a Discord-specific MCP server for direct API access (with bot token scoping).

Integrating MCP with LangChain agents is straightforward and powerful for your local Discord bot setup. It lets your Ollama-powered agent discover and use tools/resources from MCP servers (e.g., filesystem access, vector DB queries, Discord history, external scans) in a standardized way.

This builds on your previous components: Ollama for the LLM, LangChain/LlamaIndex for orchestration/RAG, and MCP for tool exposure.

Why This Combination?

- **LangChain Agents:** Handle reasoning loops, tool selection, and multi-step tasks (ReAct, etc.).
- **MCP:** Provides discoverable, secure tools/resources without custom tool definitions for every integration.
- **Local/Ollama:** Fully offline, model-swappable (Qwen2.5, etc.), and private.
- **Benefits for Bot:** Agent can infer server activity, perform RAG via MCP resources, run scheduled tasks, or take constrained actions.

Prerequisites

- Running MCP server(s) (from previous guidance, e.g., using FastMCP).
- LangChain ecosystem:

```
pip install langchain langchain-ollama langchain-mcp-adapters langgraph # langgraph  
for advanced agents
```

Core Integration Pattern

Use MultiServerMCPClient from langchain-mcp-adapters to load MCP tools into LangChain format, then bind them to a LangChain agent.

Simple Example (agent_with_mcp.py)

```
import asyncio
from langchain_ollama import ChatOllama
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent # Or create_tool_calling_agent

async def main():
    # Connect to one or more MCP servers
    client = MultiServerMCPClient({
        "filesystem": { # Example label
            "transport": "stdio",
            "command": "uvx", # or "npx", "python"
            "args": ["mcp-server-filesystem", "/path/to/data"],
        },
        "discord_helper": { # Your custom server
            "transport": "stdio",
            "command": "python",
            "args": ["path/to/your_mcp_server.py"],
        },
        # Add more: vector DB, web fetch, etc.
    })

    # Load tools (async)
    tools = await client.get_tools()

    # Ollama model (swap easily)
    model = ChatOllama(
```



```

model="qwen2.5:7b-instruct", # or gemma2, mistral, etc.
temperature=0.7,
# num_ctx=8192, etc.
)

# Create agent
agent = create_react_agent(model, tools) # ReAct style

# Example invocation
response = await agent.ainvoke({
    "messages": [("user", "Summarize recent activity in the Discord server and check
knowledge base.")]
})
print(response)

if __name__ == "__main__":
    asyncio.run(main())

```

Key Points:

- **MultiServerMCPClient:** Handles multiple servers, tool discovery, and lifecycle.
- **Transports:** stdio (local processes) or HTTP/SSE.
- **Agent Types:** create_react_agent, create_tool_calling_agent, or custom LangGraph workflows for complex flows (e.g., with RAG).

Integrate with Discord Bot

In your discord.py bot:

- On message (e.g., mention or !agent command): Pass user input to the LangChain agent.
- Agent uses MCP tools (e.g., fetch history via Discord MCP tool, query vector store, generate overview).
- Stream response back to channel (Ollama supports streaming).
- Add memory (e.g., LangChain conversation memory or vector store) for context across interactions.
- Rate limiting: Wrap agent calls with queues/delays.

Example snippet:

```

@client.event
async def on_message(message):
    if ... : # trigger condition

```

```
# Run agent asynchronously
result = await run_agent(message.content)
await message.channel.send(result)
```

Combining with RAG

- Expose your Chroma/LlamaIndex index as an MCP resource/tool (e.g., `semantic_search(query)`).
- Agent decides when to use RAG tool vs. others.
- Scheduled jobs can update the index and expose fresh data via MCP.

Advanced Tips

- **Model Compatibility:** Tool-calling works best with stronger models (Qwen2.5, Mistral variants). Test smaller ones with explicit prompts.
- **LangGraph:** For stateful multi-agent setups (e.g., one agent for analysis, another for response).
- **Error Handling & Safety:** Add approval steps for sensitive tools. Use `require_approval` where supported. Handle Ollama errors gracefully.
- **Performance:** Run MCP servers and agent in the same process or via background tasks. Monitor resources.
- **Customization:** Build MCP tools that call Discord API (with your bot token), query Postgres/Chroma, or run scheduled inferences.
- **Debugging:** Use logging. Test MCP tools standalone before agent integration.

Resources

- LangChain MCP docs/adapters.
- Examples: GitHub repos combining LangChain + MCP + Ollama (search for demos with `MultiServerMCPClient`).
- Videos/tutorials on building MCP client/server with LangChain agents.

This creates a highly capable, modular local agent for your Discord server—able to reason, retrieve, act, and adapt. Start with a simple math/filesystem MCP server + basic agent, then layer in your custom Discord/RAG tools.

LangGraph state management is essential for building reliable, conversational, and multi-step agents in your local Discord bot. It provides **short-term memory** (conversation history within a thread) via checkpoints and supports **long-term memory** through stores or custom databases.

This fits your stack: Ollama + MCP tools + LangChain agents, enabling the bot to remember server context, user preferences, past inferences, and ongoing tasks across interactions.

Key Concepts in LangGraph State

- **State Schema:** Typed (often TypedDict or Pydantic) definition of what the graph tracks (messages, metadata, RAG results, etc.).
- **Checkpointers:** Persist snapshots of state at each node/step. Enables resuming, human-in-the-loop, and recovery.
- **Threads:** Unique conversation IDs (e.g., per Discord user/channel). State is isolated per thread.
- **Persistence:** SQLite (easy local), Postgres (production-like), Redis, etc.
- **Stores:** For long-term/cross-thread memory (e.g., user profiles, server summaries).

Implementation Plan

Install Dependencies

```
pip install langgraph langgraph-checkpoint-sqlite langchain-ollama
langchain-mcp-adapters
```

Define State

```
from typing import TypedDict, Annotated, Sequence
from langgraph.graph.message import add_messages
from langchain_core.messages import BaseMessage

class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], add_messages] #
    # Auto-appends messages
    server_context: str # e.g., recent activity summary
    rag_results: list # Or any custom data
    user_preferences: dict # Long-term info
    # Add MCP tool results, etc.
```

Build the Graph with Persistence

```
import asyncio
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.sqlite import SqliteSaver # Or AsyncSqliteSaver
from langchain_ollama import ChatOllama
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.prebuilt import create_react_agent # Or custom nodes

# Checkpointer (persistent)
```

```

memory = SqliteSaver.from_conn_string("checkpoints.sqlite") # File for persistence
# memory = SqliteSaver.from_conn_string(":memory:") # Testing only

# MCP tools
async def get_tools():
    client = MultiServerMCPClient({...}) # Your MCP config
    return await client.get_tools()

# LLM
llm = ChatOllama(model="qwen2.5:7b-instruct", temperature=0.7)

# Build graph (simple ReAct or custom)
async def build_agent():
    tools = await get_tools()
    graph = create_react_agent(llm, tools, checkpointer=memory) # Or StateGraph
    for full control
    return graph

# Usage in Discord bot
async def handle_message(user_id: str, channel_id: str, user_input: str):
    thread_id = f"{user_id}-{channel_id}" # Unique per conversation

    config = {"configurable": {"thread_id": thread_id}}

    agent = await build_agent() # Or compile once globally

    # Invoke with state persistence
    result = await agent.ainvoke(
        {"messages": [("user", user_input)]},
        config=config
    )

    # Extract response
    return result["messages"][-1].content

```

How it works:

- First call creates a new thread/checkpoint.
- Subsequent calls in the same thread_id resume state (history, context).
- Checkpoints save after each node (tool call, LLM response).

Advanced: Custom StateGraph + Long-Term Memory

For more control (e.g., scheduled summaries, MCP integration in nodes):

```
workflow = StateGraph(AgentState)

def agent_node(state: AgentState):
    # Call LLM with tools/MCP
    ...

def summarize_node(state: AgentState):
    # Use MCP or Ollama to update server_context
    ...

workflow.add_node("agent", agent_node)
workflow.add_node("summarize", summarize_node)
# Edges, conditional routing, etc.

app = workflow.compile(checkpointer=memory)
```

Long-term memory: Use InMemoryStore or PostgresStore + semantic search for cross-thread recall (e.g., user history across servers).

Discord Bot Integration

- On bot startup: Load or initialize the compiled graph.
- Use `thread_id` based on Discord user/channel/thread ID.
- Handle interruptions: Discord messages can resume threads naturally.
- Background tasks (APScheduler): Periodically invoke graph nodes for indexing/summaries, updating shared state.

Multi-user handling: Each interaction specifies its `thread_id`—LangGraph isolates state automatically.

Persistence Options

- **SQLite:** Great start for local (checkpoints.sqlite file). Simple, no server needed.
- **Postgres:** For heavier use (langgraph-checkpoint-postgres).
- **Redis:** Excellent for speed + persistence.
- Prune old checkpoints periodically for storage management.

Best Practices for Your Bot

- **Thread Strategy:** Per-user, per-channel, or per-conversation threads depending on privacy/scope.
- **State Size:** Keep it lean; offload heavy data (embeddings) to vector DB + MCP.
- **Error Recovery:** Checkpoints allow resuming failed runs.
- **Testing:** Use `get_state()` and `get_state_history()` to inspect/debug.
- **Security:** Sanitize inputs; scope MCP tools carefully.
- **Performance:** Async everywhere. Test with your smaller models first.

This turns your bot into a stateful assistant that remembers server activity, user interactions, and builds cumulative knowledge. Combine with your RAG pipeline and MCP tools for powerful inferences/overviews.

Custom state graph nodes give you full control in LangGraph for complex workflows in your Discord bot (e.g., MCP tool orchestration, RAG retrieval, server activity analysis, summarization, or scheduled inferences).

This builds on the previous state management setup. Custom nodes are Python async functions (or sync) that take the current state and return updates.

Recommended Structure

Define a typed state, build a StateGraph, add custom nodes with edges/conditional routing, and compile with a checkpointer.

State Definition (state.py)

```
from typing import TypedDict, Annotated, Sequence, Optional
from langgraph.graph.message import add_messages
from langchain_core.messages import BaseMessage

class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], add_messages]
    server_context: str # Accumulated server overview / inferences
    rag_context: list[dict] # Retrieved documents
    mcp_results: dict # Results from MCP tools
    user_id: str
    channel_id: str
    metadata: dict # Any extra (preferences, timestamps)
```

Custom Nodes (nodes.py)

```
import asyncio
```

```

from langchain_ollama import ChatOllama
from langchain_mcp_adapters.client import MultiServerMCPClient
# Assume you have RAG setup (LlamaIndex/Chroma)
from your_rag import retrieve_relevant_docs

llm = ChatOllama(model="qwen2.5:7b-instruct", temperature=0.7)

async def retrieve_mcp_tools():
    client = MultiServerMCPClient({ ... }) # Your servers (filesystem, discord_helper,
    etc.)
    return await client.get_tools()

# Node 1: Retrieve Context (RAG + MCP)
async def context_retriever(state: AgentState) -> AgentState:
    query = state["messages"][-1].content if state["messages"] else ""

    # RAG retrieval
    rag_docs = retrieve_relevant_docs(query) # Your Chroma/LlamaIndex function

    # MCP tools discovery / pre-call if needed
    mcp_tools = await retrieve_mcp_tools()

    return {
        "rag_context": rag_docs,
        "mcp_results": {}, # Populated later
    }

# Node 2: Agent Reasoning + Tool Use (with MCP)
async def agent_node(state: AgentState) -> AgentState:
    tools = await retrieve_mcp_tools() # Or cache globally

    # Bind tools to LLM (LangChain handles calling MCP via adapters)
    bound_llm = llm.bind_tools(tools)

    # Invoke with full context
    prompt_context = f"Server context: {state.get('server_context', '')}\nRAG: {state.get('rag_context', [])}"

    response = await bound_llm.ainvoke(state["messages"] + [prompt_context])

    return {"messages": [response]}

# Node 3: Post-Processing / Summarization
async def summarizer_node(state: AgentState) -> AgentState:

```

```

last_message = state["messages"][-1].content
summary_prompt = f"Summarize recent activity and key points: {last_message}"

summary = await llm.ainvoke(summary_prompt)

# Update persistent server context
new_context = f'{{state.get('server_context', '')}}\n{summary.content}'

return {
    "server_context": new_context[:2000], # Truncate if needed
    "messages": [summary] # Optional: add to messages
}

# Node 4: Example Custom Action (e.g., scheduled inference or external scan)
async def external_scan_node(state: AgentState) -> AgentState:
    # Call MCP tool for scanning resources or DB query
    # Example: assume an MCP tool "scan_resources"
    # result = await some_mcp_tool(...)
    return {"metadata": {"last_scan": "timestamp"}}

```

Build the Graph (graph.py)

```

from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.sqlite import SqliteSaver
from nodes import context_retriever, agent_node, summarizer_node,
external_scan_node

memory = SqliteSaver.from_conn_string("checkpoints.sqlite")

workflow = StateGraph(AgentState)

workflow.add_node("retrieve_context", context_retriever)
workflow.add_node("agent", agent_node)
workflow.add_node("summarize", summarizer_node)
workflow.add_node("scan", external_scan_node)

# Edges
workflow.add_edge(START, "retrieve_context")
workflow.add_edge("retrieve_context", "agent")
workflow.add_edge("agent", "summarize")

# Conditional routing (example)
def route_after_summary(state: AgentState):
    # Decide next action based on state

```



```

        if "scan_needed" in state.get("metadata", {}):
            return "scan"
        return END

workflow.add_conditional_edges("summarize", route_after_summary)

app = workflow.compile(checkpointer=memory)

```

Usage in Discord Bot

```

async def handle_discord_message(message):
    thread_id = f"user_{message.author.id}_chan_{message.channel.id}"
    config = {"configurable": {"thread_id": thread_id}}

    input_state = {
        "messages": [("user", message.content)],
        "user_id": str(message.author.id),
        "channel_id": str(message.channel.id),
    }

    result = await app.ainvoke(input_state, config=config)

    response = result["messages"][-1].content
    await message.channel.send(response)

# Optional: Trigger background updates

```

Key Benefits & Tips

- **Modularity:** Each node is testable independently (pass mock state).
- **MCP Integration:** Load tools once or per-node; adapters make them LangChain-compatible.
- **RAG:** Inject retrieved context into the agent node or a dedicated retriever node.
- **Persistence:** State (including custom fields like `server_context`) survives restarts via the checkpointer.
- **Conditional Logic:** Use `add_conditional_edges` for branching (e.g., escalate to human, run scan, or end).
- **Error Handling:** Wrap nodes in `try/except`; `LangGraph` supports retries via configuration.
- **Performance:** Use `async` nodes. Cache heavy objects (tools, embeddings) outside the graph if possible.
- **Debugging:** `app.get_state(config)` and `app.get_state_history(config)` to inspect.

Extensions for Your Bot

- **Scheduled Work:** Separate graph or node invoked by APScheduler to update server_context or index data via MCP.
- **Multi-Agent:** Sub-graphs (one for analysis, one for response).
- **Long-term Memory:** Add a Store for cross-thread user/server knowledge.

This creates a flexible, stateful agent that can retrieve context, reason with MCP tools, summarize activity, and maintain overview over time. Test incrementally: start with 2-3 nodes, then expand.